Q5. Heuristic vs Non-Heuristic Search, and Simulated Annealing for Product Recommendation

> **Question.** Discuss heuristic and non-heuristic search approaches. In product recommendation you want to use the simulated annealing (SA) search algorithm to recommend products. How will you use it? Explain in detail with an example.

1. The idea in plain words

A recommender has to pick a few products, a "slate", from a big catalogue. Trying every slate is too slow, so we **search**. Plain hill climbing only accepts changes that make the slate better, so it gets stuck on the first good-looking slate. Simulated annealing behaves like a curious shopper: early on (when it is "hot") it sometimes accepts a *worse* slate just to look around. As it "cools down" it becomes picky and settles on the best slate it found.

2. Heuristic vs non-heuristic search

Non-heuristic (uninformed, blind) search uses only the problem definition: start, actions and goal test. It has no idea which direction is "warmer". **Heuristic (informed) search** also uses a function h(n) or a score that estimates how good or how close a state is, so it can try promising states first.

Type	Algorithm	What guides it	Complete?	Optimal?	Memory
Non-heuristic	Breadth-first (BFS)	FIFO queue, level by level	Yes	Yes, if step costs are equal	High, $O(b^d)$
	Depth-first (DFS)	LIFO stack, deepest first	No	No	Low, O(bm)
	Uniform cost (UCS)	Lowest path cost g(n)	Yes (step cost > 0)	Yes	High
	Iterative deepening	DFS with a growing depth limit	Yes	Yes, for unit costs	Low
Heuristic	Greedy best-first	Lowest h(n) only	No	No	High
	A*	f(n) = g(n) + h(n)	Yes	Yes, if h is admissible	High
	Hill climbing	Best neighbour only	No (local maxima)	No	Very low (1 state)
	Simulated annealing	Score + random worse moves, cooling T	Yes, in the limit of slow cooling	Probabilistically	Very low (1 state)

For recommendations the "path" does not matter; only the final slate does. That makes **local search** (hill climbing, SA) a natural fit: it keeps one slate in memory and keeps improving it.

3. Assumptions

- A catalogue of **16 products** in 9 categories. Each product has an interest vector (audio, fitness, photo, work) and a popularity value.
- One user who likes audio, fitness and photography, and who **has been browsing phones all week**.

- We show a slate of **k = 3** products. That gives C(16,3) = 560 possible slates, small enough to check the true best by brute force.

4. Mapping SA to recommendation

SA term	Meaning here
State	A slate of 3 products for this user
Neighbour	Swap one product in the slate for one outside it
Score (energy)	score = mean relevance + 0.2 × mean novelty + 0.25 variety bonus
Relevance	0.7 × cosine(user, product) + 0.3 if the product is a phone (browsing history)
Novelty	1 − popularity (less famous items feel fresher)
Variety bonus	+0.25 only when all 3 products come from different categories
Temperature T	How willing we are to try a worse slate. Starts at $T_0 = 0.10$
Acceptance	Better slate: always. Worse slate: with probability $p = e^{\Delta/T}$
Cooling	$T \leftarrow 0.97 \times T$ after every step, for 250 steps

5. How to use SA in a recommender (step by step)

1. Build the starting slate from what the user already looks at (here, their 3 most-viewed phones).
2. Set a high starting temperature. This is the exploring phase.
3. Make a random neighbour: swap one product.
4. If the new slate scores higher, keep it. If it scores lower, keep it anyway with probability $e^{\Delta/T}$.
5. Cool down ($T \leftarrow \alpha T$) and repeat. Remember the best slate seen so far.
6. When T is tiny, show the best slate. Re-run it every session, because the user's history changes.

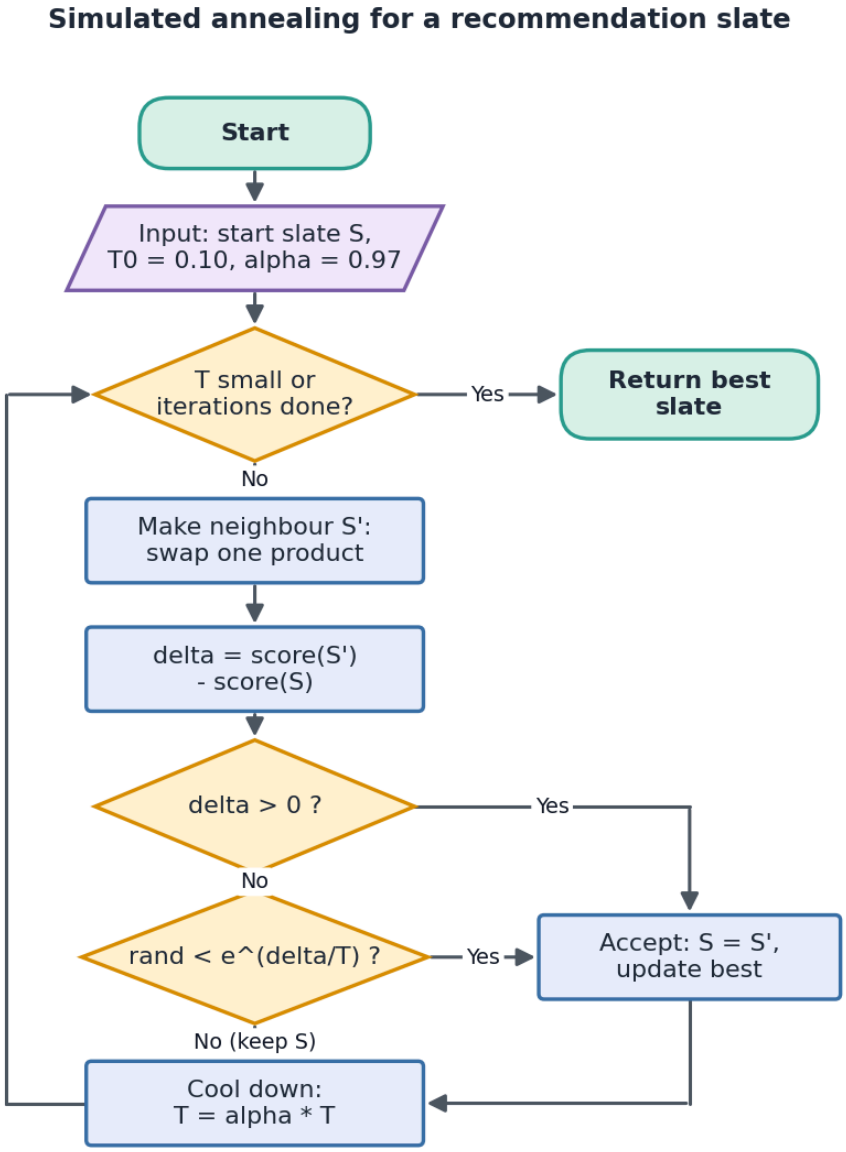


Figure 1. Flowchart of simulated annealing for a recommendation slate.

6. Pseudocode

```
function SA_Recommend(start_slate, T0 = 0.10, alpha = 0.97, iters = 250):
    current ← start_slate;  best ← current;  T ← T0
    for it = 1 to iters:
        out ← random product in current
        inn ← random product not in current
        candidate ← current with out replaced by inn          # neighbour
        delta ← score(candidate) − score(current)
        if delta > 0 or random() < e^(delta / T):          # worse moves allowed while hot
            current ← candidate
        if score(current) > score(best): best ← current
        T ← alpha × T                                      # gradual cooling
    return best
```

7. Worked example (real run)

Hill climbing gets stuck on "all phones"

The start slate is **Phone Pro X, Phone Max, Phone Lite** with score **0.961**. Hill climbing makes one good swap (Phone Max → Phone Mini, score **1.007**) and then stops. Every single swap from there is worse. Taking out one phone loses a lot of relevance, and with only two categories there is no variety bonus yet. This is a **local maximum**: a "same-category" bubble.

SA walks out of the bubble

Iter	T	Move tried	Δ	p	Result	Score now
1	0.100	Phone Pro X → Action Cam	−0.058	0.56	accepted (worse!)	0.903
2	0.097	Phone Lite → Home Speaker	+0.073	1.00	accepted	0.976
3	0.094	Action Cam → Phone Lite	−0.174	0.16	rejected	0.976
4	**0.091**	**Home Speaker → SportBuds**	**+0.060**	**1.00**	**accepted**	**1.036**

At iteration 1 SA accepts a **worse** slate ($\Delta = -0.058$, $p = e^{-0.058/0.10} \approx 0.56$). That single "step down" lets it reach three different categories at iteration 2 and earn the variety bonus. By iteration 4 it is at 1.036, already above the hill-climbing peak of 1.007. A very bad move at iteration 3 ($\Delta = -0.174$) had only a 16% chance and was rejected.

Slate	Mean relevance	0.2 × novelty	Variety	Score
Start: Pro X, Max, Lite	0.938	0.023	0	0.961
Hill-climbing peak: Pro X, Lite, Mini	0.964	0.043	0	1.007
SA result: Phone Mini, Tab S, Action Cam	**0.734**	**0.100**	**0.25**	**1.084**

As T fell (0.048 at iteration 25, 0.005 at 100, 0.0001 at 250), SA accepted fewer bad moves (15 in total, almost all early). It ended on **Phone Mini, Tab S, Action Cam = 1.084**. A brute-force check of all 560 slates confirms this is the **global optimum**. The slate is still relevant (one phone the user likes) but also fresh and varied.

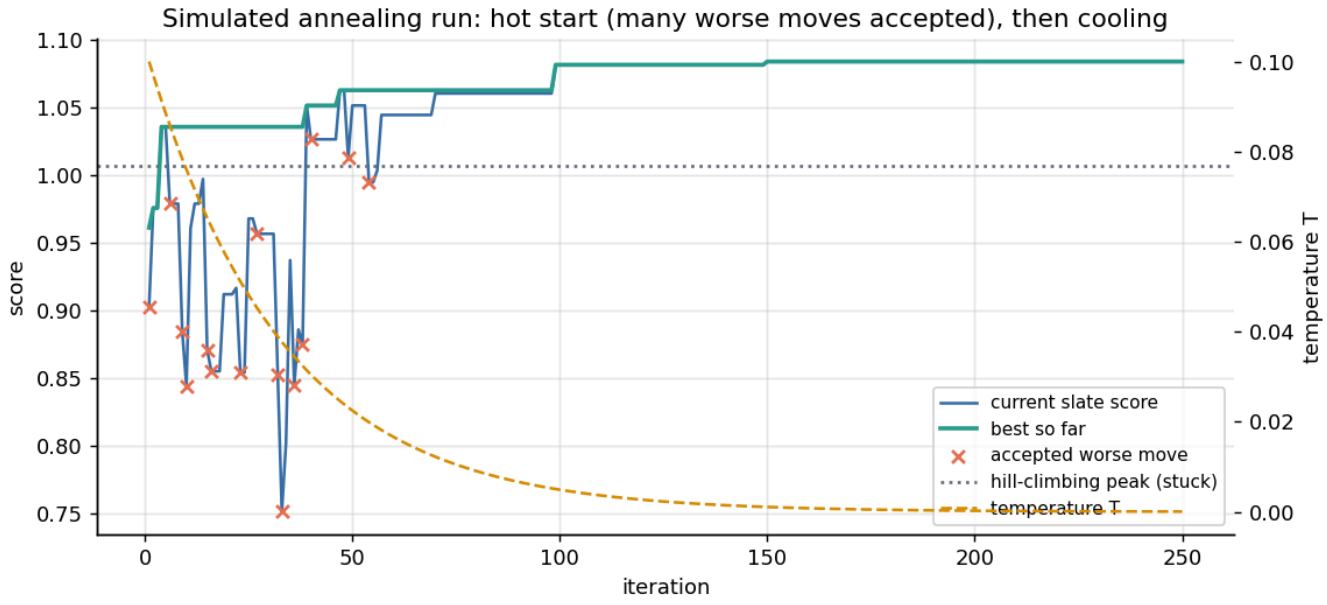


Figure 2. The real SA run. The red crosses are accepted worse moves. They happen while T is high, then die out as the system cools.

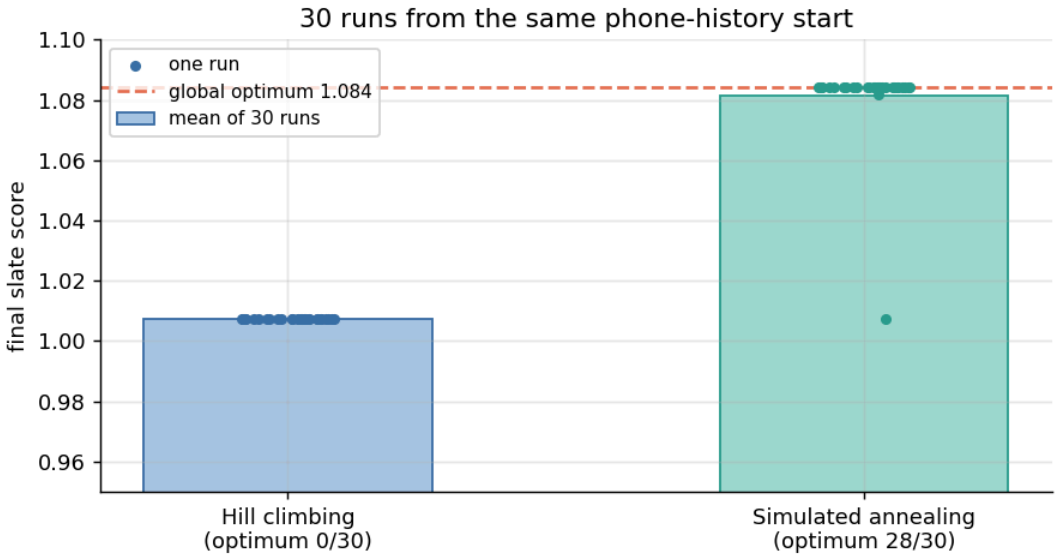


Figure 3. 30 runs from the same start. Hill climbing always stops at 1.007. SA finds the optimum in 28 of 30 runs (mean 1.082).

8. Why SA suits recommendation

- **It breaks the filter bubble.** Pure "more of the same" (hill climbing on relevance) keeps showing phones. SA's early worse moves bring in serendipity: the camera, the tablet.
- **Slow to heat and gradual cooling.** This is the course's choice principle: introduce the best choice gradually and keep it warm. A hot start explores, slow cooling exploits, and the final slate is stable.
- **It is cheap.** Only one slate is kept in memory, and each step scores a single swap. It is easy to run per user, per session.
- **Tuning matters.** If T_0 is too low, SA behaves like hill climbing. If cooling is too fast, it freezes early (2 of our 30 runs still missed the optimum). Random restarts or a slower α fix this.

9. Code and real run

`q05_code.py` scores the slates, runs hill climbing, runs SA, checks the global optimum by brute force, and repeats SA 30 times for the comparison chart.

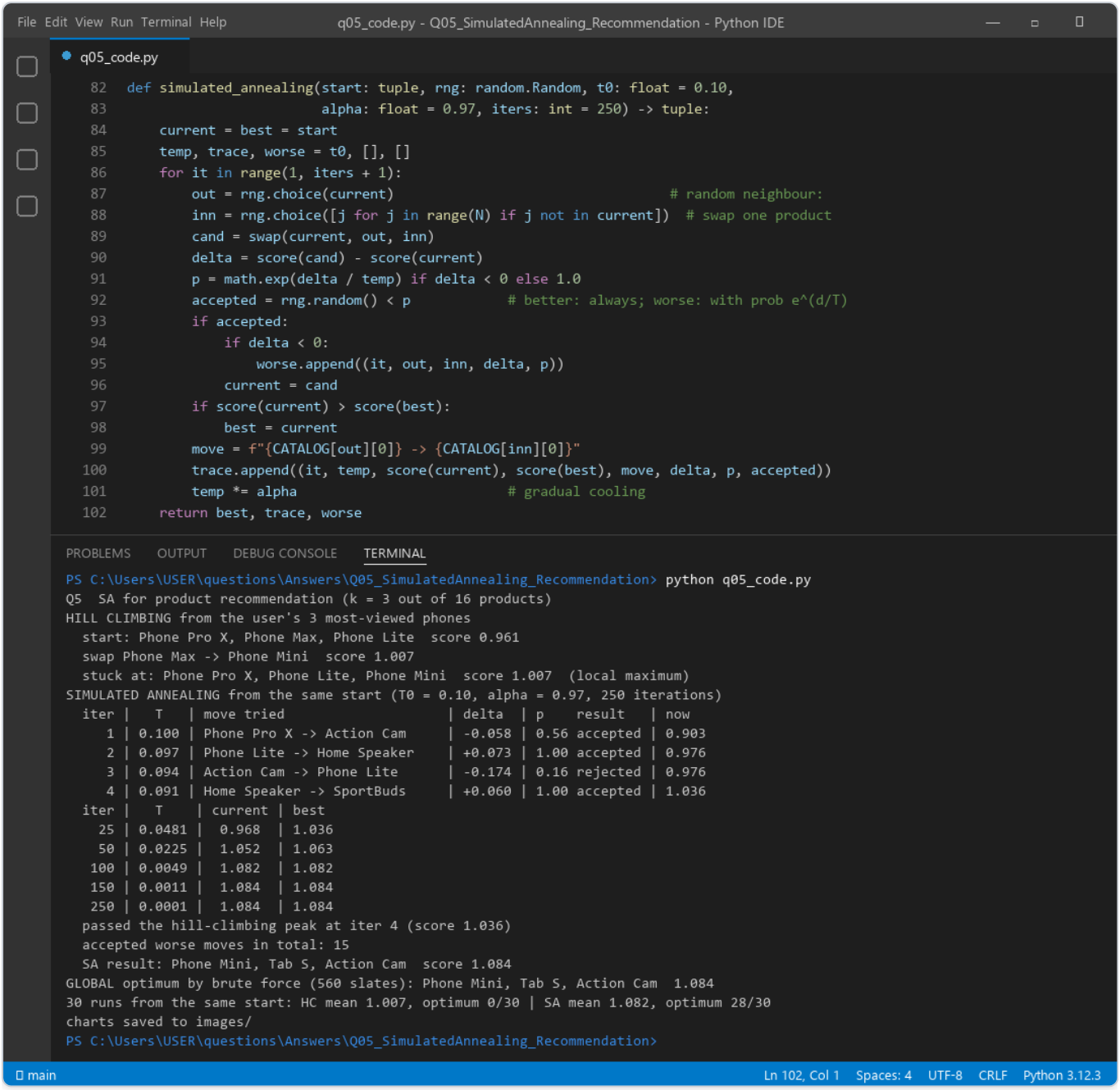


Figure 4. The SA function (lines 82-102) and the real terminal output of the full run.

10. Summary

> Non-heuristic search explores blindly. Heuristic search uses a score to head for good states. For recommendations we only care about the final slate, so local search fits. Hill climbing got trapped on an all-phones slate (1.007). Simulated annealing accepted a worse slate early (p = 0.56), escaped, and found the true best slate (1.084) in 28 of 30 runs. It mixes relevance with novelty and variety, which is exactly the creative balance a good recommender needs.

Files in this folder: `q05_code.py` (source), `q05_code.ipynb` (Colab/Jupyter version, already run), `q05_output.txt` (console output), `images/` (all figures).